AlphaFree: Recommendation Free from Users, IDs, and GNNs (Online Appendix)

Minseo Jeon
Soongsil University
Seoul, Republic of Korea
minseojeon@soongsil.ac.kr

Junwoo Jung
Soongsil University
Seoul, Republic of Korea
junwoojung@soongsil.ac.kr

Daewon Gwak
Soongsil University
Seoul, Republic of Korea
daewon1@soongsil.ac.kr

Jinhong Jung*
Soongsil University
Seoul, Republic of Korea
jinhong@ssu.ac.kr

1 Computational Complexity Analysis

Time complexity analysis. We theoretically analyze the time complexity of each phase of AlphaFree, where Algorithms 1, 2, and 3 show the preprocessing, training, and inference phases, respectively. Suppose $n = |\mathcal{I}|$ is the number of items, $m = \sum_{I \in \mathcal{H}_{\text{train}}} |I|$ is the total number of interactions, $h = |\mathcal{H}_{\text{train}}|$ is the number of interaction sets ($h \leq m$), K_c is the number of similar items ($K_c \leq n$), $|\mathcal{B}|$ is the size of mini-batch, d_{LR} is the dimension of LRs, and d denotes the embedding dimensions of the MLP layers.

Lemma 1 (Time Complexity for Preprocessing). *The time complexity of Algorithm 1 is $O(T_{\text{LM}}n + mn + (d_{\text{LR}} + \log K_c)n^2)$, where T_{LM} denotes the time at which the LM produces an embedding.*

Proof. The algorithm takes $O(T_{\text{LM}}n)$ time as it computes the LRs of all items (line 1). Constructing an inverted list for $H(i)$ takes $O(m)$ time, since $H(i)$ for each i is built by iterating over all interactions in $\mathcal{H}_{\text{train}}$ (line 2). Each intersection for $\texttt{sim}_B(\cdot)$ takes $O(|H(i)| + |H(j)|)$ time, assuming each $H(i)$ is sorted, and then it takes $\sum_{i \leq j}(|H(i)| + |H(j)|) = (n+1)\sum_i |H(i)| = O(nm)$ time (line 6). Computing all $\texttt{sim}_S(\cdot)$ takes $O(d_{\text{LR}}n^2)$. Then, it consumes $O(n^2 \log K_c + nK_c)$ time to find similar items $\mathcal{S}_i$ for all items (line 8), if we use min-heap for sorting. It takes $O(mK_c)$ and $O(nK_c d_{\text{LR}})$ for obtaining I^+ and $\mathbf{z}_i^+$, respectively (lines 9 - 11). Combining all the above terms proves the claim. ∎

Lemma 2 (Time Complexity for Training). *In Algorithm 2, each epoch takes $O((T_i dn + K_c m + dh)d_{\text{LR}} + (n_s + |\mathcal{B}|)hd)$ time, where $T_i = h/|\mathcal{B}|$ is the number of iterations over all batches, and n_s is the number of negatives.*

Proof. We first analyze the time for processing each mini-batch $\mathcal{B}$. It takes $O(d_{\text{LR}}dn)$ time to perform the MLP operations for all items (line 4). It takes $O(d_{\text{LR}}(|I|+d))$ time for each I and $O(d_{\text{LR}}(|I^+|+d))$ for each I^+ (lines 6 and 7), where $|I^+| = O(K_c|I|)$, thus the overall cost becomes $O(d_{\text{LR}}(K_c|I| + d))$. Sampling p and N_I takes $O(n_s)$ time in expectation when using rejection sampling (line 8). Then, it takes $O(|\mathcal{B}|n_s d)$ time for $\mathcal{L}_{\text{rec}}$, and $O(|\mathcal{B}|^2 d)$ time for $\mathcal{L}_{\text{align}}$ (lines 9 and 10). Considering all batches is represented as follows:

$$\begin{aligned}
&\sum_{\mathcal{B} \subseteq \mathcal{H}_{\text{train}}} \Big(d_{\text{LR}}dn + \sum_{I \in \mathcal{B}} d_{\text{LR}}(K_c|I| + d) + n_s + |\mathcal{B}|n_s d + |\mathcal{B}|^2 d \Big) \\
&\quad = T_i d_{\text{LR}}dn + d_{\text{LR}}K_c m + d_{\text{LR}}dh + T_i n_s + hn_s d + |\mathcal{B}|hd \\
&\quad = O((T_i dn + K_c m + dh)d_{\text{LR}} + (n_s + |\mathcal{B}|)hd,
\end{aligned}$$

where $n_s \leq n$. ∎

*Corresponding author.

Lemma 3 (Time Complexity for Inference). *The time complexity of Algorithm 3 is $O(d_{\text{LR}}dn + n \log K)$, where K is the parameter for top-K recommendation.*

Proof. The algorithm takes $O(d_{\text{LR}}(|I_q| + d))$ time to perform the averaging and MLP operations (line 1). For all items, it takes $O(d_{\text{LR}}dn)$ for computing their embeddings (line 2). Then, it takes $O(dn + n \log K)$ time to compute the scores of all items and select the top-K items using a min-heap (line 3). Combining all the above terms proves the claim. ∎

Space complexity analysis. We analyze the space complexity of each phase.

Lemma 4 (Space Complexity for Preprocessing). *The preprocessing phase of AlphaFree consumes $O(d_{\text{LR}}n + K_c(m+n))$ space.*

Proof. The LRs $\{\mathbf{z}_i\}$ and $\{\mathbf{z}_i^+\}$ for all items i require $O(d_{\text{LR}}n)$ space (lines 1 and 11). The $\{H(i)\}$ for all items uses $O(m)$ space (line 2). The set I^+ uses $O(K_c|I|)$ space, and thus for all $I \in \mathcal{H}_{\text{train}}$, it takes $O(K_c m)$ space (line 10). To find and store $\mathcal{S}_i$, it uses $O(K_c)$ space when a heap is used, and $O(K_c n)$ in total for all items (line 8). Note that for each i, it computes similarity scores of i for every j in an in-place manner, requiring only $O(n)$ space without storing results from previous i's (lines 5-7). Combining all the above terms proves the claim. ∎

Lemma 5 (Space Complexity for Training). *The training phase of AlphaFree uses $O(m + d(n + d_{\text{LR}} + n_s + |\mathcal{B}|))$ space.*

Proof. As an input, $\mathcal{H}_{\text{train}}$ uses $O(m)$ space. Each MLP layer takes $O(d_{\text{LR}}d)$ space (line 1). The hidden embeddings $\{\mathbf{e}_i\}$ and $\{\mathbf{e}_i^+\}$ use $O(dn)$ space (line 4). For line 9, it uses $O(dn_s)$ space for $\mathcal{E}_I$ and $\mathcal{E}_I^+$ used to compute the local losses for $\mathcal{L}_{\text{rec}}$ in an in-place manner. For line 10, it uses $O(|\mathcal{B}|d)$ space for $\mathcal{E}_{\mathcal{B}}$, $\mathcal{E}_{\mathcal{B}}^+$, $\mathcal{E}_{P_{\mathcal{B}}}$, and $\mathcal{E}_{P_{\mathcal{B}}}^+$ used to compute the local losses for $\mathcal{L}_{\text{align}}$ in an in-place manner. Combining all the above terms proves the claim. ∎

Lemma 6 (Space Complexity for Inference). *The inference phase of AlphaFree uses $O((d_{\text{LR}} + d)n + K)$ space.*

Proof. Computing all $\{\mathbf{e}_i\}$ as well as $\mathbf{e}_{I_q}$ takes $O((d_{\text{LR}} + d)n)$ space (line 2), while selecting the top-K items requires $O(K)$ space if a heap is used (line 3). ∎

2 Case Study for Over-Smoothing

Figure 1 shows a case study comparing AlphaRec and AlphaFree to investigate the over-smoothing issue on the Baby dataset. For the heavy user (User A) with many interactions ($|I_A| = 191$), AlphaRec assigns uniformly high scores with very low variance, suggesting that its item embeddings have become excessively similar (i.e.,

over-smoothed) due to GNNs. However, AlphaFree constructs user representations without GNNs, thereby preserving embedding diversity and avoiding over-smoothing for User A. For the light user (User B) with few interactions ($|I_B| = 17$), AlphaRec yields higher yet broadly distributed scores than AlphaFree, indicating that even under sparse interactions, it produces more similar but unstable embeddings, whereas AlphaFree shows less similarity with a more compact and stable distribution, suggesting robustness against such effects.

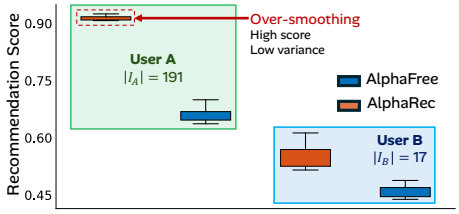

Figure 1: Case study of over-smoothing of AlphaRec and AlphaFree for heavy and light users on Baby.